# Andrew Pols

(519) 818-1374 | andrew.pols@mail.utoronto.ca | github.com/andrewpols

andrewpols.com | linkedin.com/in/andrewpols

## Education

### University of Toronto
**Expected Graduation: May 2028**

- Bachelor of Science in **Computer Science and Statistics** **GPA: 4.00 / 4.00**
- Coursework: Software Design, Computer Organization, Data Science, Software Tools and Systems Programming

## Experience

### IBM
**May 2026 – May 2027**

*Software Developer Intern*

- Shipped a dependency graph feature in React and TypeScript for Planning Analytics Workspace, which clients had asked for and competitors already offered, **automating relationship parsing** across client models with thousands of objects.
- Cut initial load on a **2,000-object** model from 30s and 50 requests to **3s and 1 request** by adding lazy-loading, request aggregation, and caching to a recursive TM1 REST API traversal; each later dependency loads in **under 500ms**.
- Translated the manual trace workflow into layout, filtering, and interaction specs with design, PM, QA, and engineering.
- Demoed to IBM leadership; later presented at the **TM1 Anwendertag** and **IBM TechXChange** conferences.
- Integrating LLM-based localization into text widgets so clients in other locales see saved text in their language.

## Projects

**ScholarSearch: Academic Search Engine** | *GitHub* Python | FastAPI | PostgreSQL | ChromaDB | Redis | React

- Engineered a hybrid BM25 and semantic search engine over 1M DBLP papers with FastAPI and ChromaDB, serving cold queries in a **334ms** median (p99 under 600ms) and cached queries in **1.2ms**.
- Embedded all 1M papers at **313 papers/sec**, then profiled the query slowdown to a 300ms in-memory BM25 scan.
- Visualized citations with a D3.js graph in React that shows how the papers behind a search cite each other.

**Multiplayer Pokémon Battle Server** | *GitHub* C | TCP Sockets | UNIX | ncurses

- Architected a Pokémon battle server in C that handles **1,000 concurrent TCP clients** in a single-threaded select() loop.
- Sustained **54k messages/sec** inbound with a **25ms p99** latency on loopback and zero connection failures.
- Defined a binary protocol with a 5-byte header and buffered reads so fragmented TCP packets parse correctly.
- Implemented 1v1 matchmaking with a lobby, rematches, forfeits, live chat, disconnect handling, and an ncurses client.

**SudokuCV: Computer Vision Sudoku Solver** | *GitHub* Java | OpenCV | Tesseract OCR

- Built a Java app that solves Sudoku from an image or webcam feed using OpenCV, Tesseract OCR, and backtracking.
- Detected the grid in 2ms per frame and solved **27 of 27** test puzzles with a 0.5ms median solve time.
- Raised exact grid reads from 7% to **92%** and digit accuracy from 89% to **99.96%** across 789 phone-video frames of 3 printed puzzles by fixing Hough-based line removal that erased stacked digits.

**FindMySound: Music Recommender** | *GitHub* Python | Django | SQLite | Scikit-Learn | React

- Developed a web app that recommends music from a user's Spotify library using the Spotify Web API and OAuth 2.0.
- Ranked songs by cosine similarity on BERT features, reaching **95% satisfaction** across 5,000 songs for 50 users.
- Cached each song's audio analysis, cutting per-candidate scoring from 2s to **0.2ms**.
- Designed the Django backend with a custom user model, JWT authentication, SQLite, and REST API endpoints.

## Technical Skills

- **Programming Languages**: Python, TypeScript, JavaScript, Java, C, Go, SQL, HTML5, CSS
- **Frameworks & Libraries**: React, Node.js, FastAPI, Django, Flask, Next.js, Scikit-learn, OpenCV, REST, GraphQL
- **Databases & Cloud**: PostgreSQL, MySQL, MongoDB, Redis, ChromaDB; AWS, Azure
- **Developer Tools**: Git, Docker, Vite, Jenkins, UNIX, Make, Valgrind, AddressSanitizer, Jira